

Data Engineering Interview — Python & SQL Theory Answers



PYTHON THEORY + REAL-LIFE DATA ENGINEERING ANSWERS

1. Mutable vs Immutable

Mutable objects can be modified after creation, while immutable objects cannot. Lists and dictionaries are mutable; tuples and strings are immutable.

DE Use Case: Immutable objects are safer for consistent processing, while mutable structures help during ETL transformations.

2. List vs Tuple vs Set vs Dictionary

- List → ordered and mutable
- Tuple → ordered and immutable
- Set → unordered unique values
- Dictionary → key-value structure

DE Use Case: Dictionaries are heavily used for JSON/API payload processing.

3. Deep Copy vs Shallow Copy

Shallow copy copies references of nested objects, while deep copy creates fully independent objects.

DE Use Case: Useful when modifying nested ETL records without affecting original datasets.

4. Iterators vs Generators

Iterator traverses elements one-by-one. Generator uses yield and creates values lazily.

DE Use Case: Generators are memory efficient for processing large files.

5. Decorators

Decorators wrap another function to add functionality.

DE Use Case: Used for logging, retry mechanisms, and monitoring pipeline functions.

6. *args* and **kwargs*

args handles variable positional arguments, while **kwargs* handles keyword arguments.

DE Use Case: Useful for reusable ingestion utilities.

7. Lambda, map(), filter(), reduce()

Functional programming utilities for concise transformations.

DE Use Case: Useful in lightweight transformations before Spark processing.

8. Exception Handling

Used to handle runtime failures gracefully.

DE Use Case: Prevents ETL pipeline crashes due to malformed records or API failures.

9. File Handling & Context Manager

Using with open() ensures automatic file closing.

DE Use Case: Important for large-scale file ingestion pipelines.

10. OOP Concepts

Class-based design improves modularity and reusability.

DE Use Case: Useful for building scalable ingestion frameworks.

11. Multithreading vs Multiprocessing

- Multithreading → good for I/O tasks
- Multiprocessing → good for CPU-heavy tasks

DE Use Case: Parallel file/API ingestion.

12. Memory Optimization & Generators

Generators avoid loading complete data into memory.

DE Use Case: Useful while processing GB-level datasets.

13. JSON Handling & API Parsing

Python dictionaries help process nested API payloads.

DE Use Case: API ingestion pipelines.

14. Virtual Environment & Requirements

Virtual environments isolate dependencies.

DE Use Case: Prevents deployment conflicts across ETL services.

15. Modular Project Structure & Logging

Separating configs, utils, ingestion, transformations improves maintainability.

DE Use Case: Production-grade ETL architecture.

16. Processing Large CSV Files

Use chunking or generators to avoid memory overflow.

17. Handling API Pagination

Loop through paginated responses using page tokens or offsets.

18. Retry Mechanism for Failed Calls

Implement retries with exponential backoff.

19. Logging Pipeline Failures

Use structured logging for debugging and monitoring.

20. YAML/JSON Config Pipelines

Metadata-driven design avoids hardcoding.

21. Reusable ETL Utility Functions

Create shared connectors, validators, and loaders.

22. Parallel File Processing

Use multiprocessing for independent file ingestion.

23. Data Validation Before Loading

Validate nulls, duplicates, schema mismatches.

24. Incremental Ingestion Logic

Load only new/updated records using timestamps.

25. Optimizing Slow ETL Jobs

Reduce loops, batch inserts, parallelize workloads.

26. Handling Corrupt JSON Records

Use try-except and dead-letter handling.

27. Scalable Ingestion Framework Structure

Use modular architecture with configs and reusable extractors.

28. Managing Secrets Securely

Use environment variables or secret managers.

29. Streaming File/Event Processing

Continuously process incoming events instead of batch schedules.

30. Metadata-Driven Pipelines

Pipeline behavior controlled dynamically using configs.

=====

SQL THEORY + REAL-LIFE DATA ENGINEERING ANSWERS

1. Joins

Used to combine records from multiple tables.

DE Use Case: Combining transactional and dimension datasets.

2. GROUP BY vs HAVING

WHERE filters rows before grouping, HAVING filters after aggregation.

3. Window Functions

Perform calculations across rows without collapsing data.

DE Use Case: Ranking and running totals.

4. ROW_NUMBER vs RANK vs DENSE_RANK

- ROW_NUMBER → unique ranking
 - RANK → skips after ties
 - DENSE_RANK → no skipped ranks
-

5. CTE vs Subquery

CTE improves readability and modular query structure.

6. EXISTS vs IN

EXISTS is generally better for large correlated datasets.

7. UNION vs UNION ALL

UNION removes duplicates, UNION ALL keeps all rows.

8. PARTITION BY

Divides data logically within window functions.

DE Use Case: Department-wise rankings.

9. LEAD & LAG

Access next or previous row values.

DE Use Case: Time-series comparisons.

10. Deduplication Using Window Functions

Use ROW_NUMBER with PARTITION BY and latest timestamp.

11. Indexing

Improves query performance using optimized lookup structures.

12. Normalization vs Denormalization

Normalization reduces redundancy; denormalization improves analytics speed.

13. Primary Key vs Foreign Key

Primary key uniquely identifies rows; foreign key creates relationships.

14. Views vs Materialized Views

Materialized views store results physically for faster reads.

15. Query Optimization & Execution Order

Understand joins, filtering, indexing, partition pruning.

16. Deduplicating Latest Records

Use ROW_NUMBER ordered by timestamp descending.

17. Incremental Load Queries

Load only records newer than last successful run.

18. CDC Query Patterns

Capture inserts/updates/deletes only.

19. Finding Missing Records

Use LEFT JOIN or NOT EXISTS.

20. Top N Customers per Category

Use window functions with partitioning.

21. Detecting Duplicate Transactions

GROUP BY + HAVING COUNT(*) > 1.

22. SCD Type 1 vs Type 2

- Type 1 → overwrite
 - Type 2 → preserve history
-

23. Revenue Aggregation Queries

Use GROUP BY with date dimensions.

24. Finding Inactive Users

Use LEFT JOIN and NULL filtering.

25. Data Reconciliation Queries

Compare source and target counts/sums.

26. Optimizing Joins on Large Datasets

Use indexing, partitioning, broadcast joins.

27. Handling NULL Values

Use COALESCE or NULL-safe logic.

28. Partition Pruning

Query scans only relevant partitions.

29. MERGE / UPSERT Operations

Update existing rows and insert new rows together.

30. Building Fact & Dimension Tables

Fact tables store measurable metrics; dimension tables store descriptive attributes.